

Diagrama de Classes

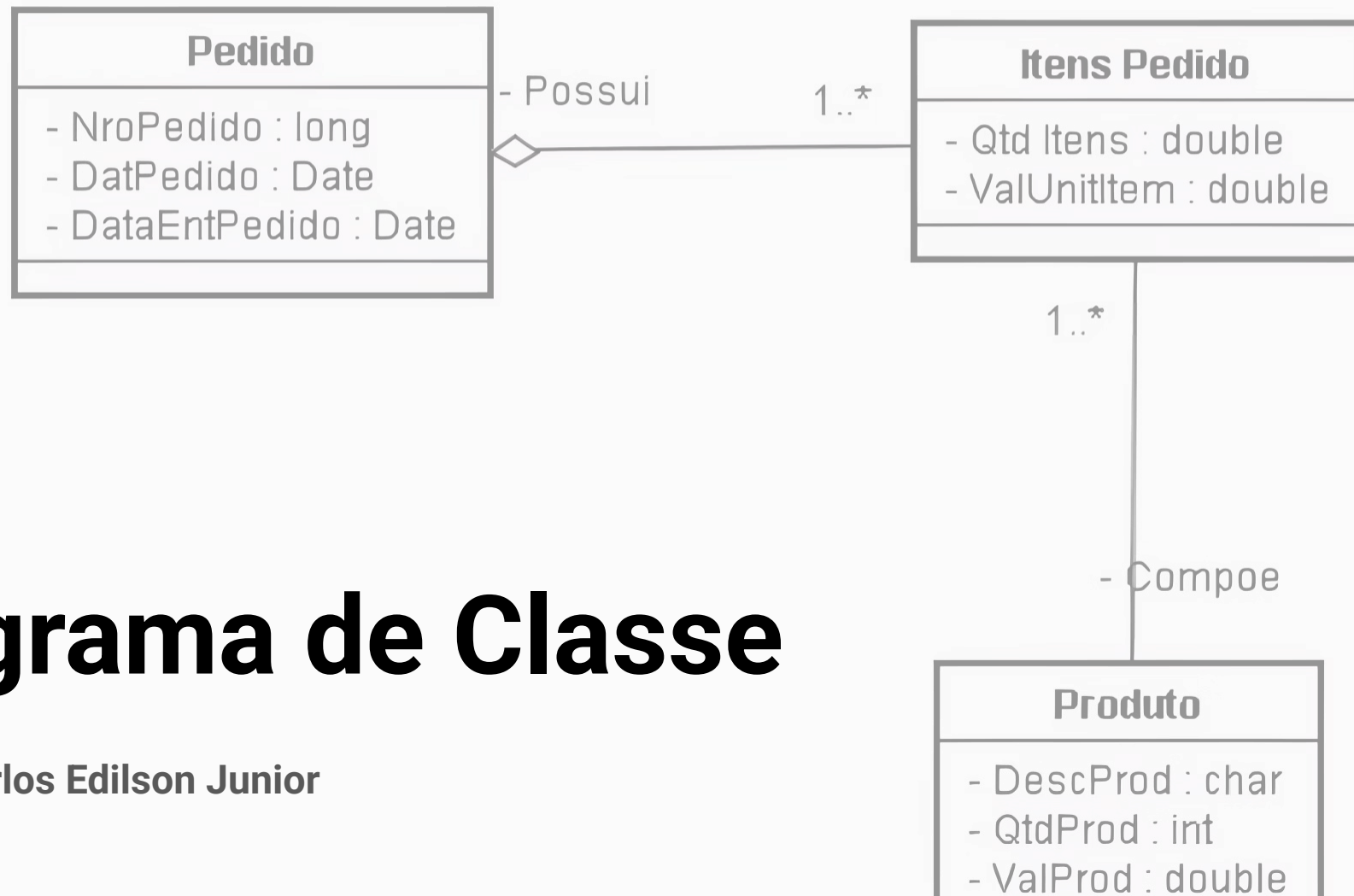


Diagrama de Classe



por Carlos Edilson Junior

O Que É um Diagrama de Classe?

O Diagrama de Classe é uma ferramenta visual fundamental na Modelagem Orientada a Objetos, parte da Linguagem de Modelagem Unificada (UML). Ele oferece uma representação clara e estática da estrutura de um sistema.

Visão Estática do Sistema

Descreve a estrutura do sistema em termos de classes, seus atributos e as operações (métodos) que elas realizam.

Relacionamentos Essenciais

Ilustra como as classes interagem entre si, incluindo associações, agregações, composições e herança.

Base para Implementação

Serve como um blueprint para desenvolvedores, facilitando a compreensão e implementação do código-fonte.

Por Que o Diagrama de Classe é Crucial?

A importância do Diagrama de Classe reside em sua capacidade de otimizar o processo de desenvolvimento de software, oferecendo benefícios chave:



Clareza Estrutural

Visualiza a arquitetura do software, facilitando a compreensão de sistemas complexos por toda a equipe.



Comunicação Unificada

Cria uma linguagem comum entre desenvolvedores, analistas e stakeholders, alinhando expectativas do projeto.



Fundamento para o Código

Guia a codificação, otimizando o desenvolvimento, a manutenção e a expansão futura do software.

Elementos Chave de um Diagrama de Classe

Para construir um Diagrama de Classe eficaz, é essencial compreender seus componentes fundamentais, que delineiam a estrutura e o comportamento do sistema.



Classes

Representam entidades do sistema com um nome, atributos e métodos que definem seu estado e comportamento.



Atributos

São as características ou propriedades que uma classe possui, descrevendo os dados e o estado de seus objetos.



Métodos (Operações)

Ações ou funções que uma classe pode executar, definindo seu comportamento e como interage com outros objetos.



Relacionamentos

Conexões entre classes (associação, herança, agregação, composição) que mostram como elas interagem entre si.

Classes

- Nome da classe
- Atributos
 - Tipo
 - Visibilidade
- Métodos
 - Parâmetros
 - Tipo de retorno
 - Visibilidade



Visibilidade de Atributos e Métodos

A visibilidade define o nível de acesso aos atributos (dados) e métodos (comportamento) de uma classe, controlando quem pode interagir com eles dentro ou fora do sistema. Existem três níveis principais:



Público (**public**)

Acessível de qualquer lugar. Atributos e métodos marcados como públicos podem ser acessados por qualquer outra classe ou objeto no sistema, facilitando a interação e a exposição de funcionalidades essenciais.



Privado (**private**)

Acessível apenas dentro da própria classe. Isso garante o encapsulamento, protegendo os detalhes internos da classe de modificações externas e mantendo a integridade dos dados.



Protegido (**protected**)

Acessível dentro da própria classe e por suas subclasses. Permite que classes herdadas acessem e modifiquem elementos, mantendo um certo nível de encapsulamento em relação a outras classes não relacionadas.

Multiplicidade no Diagrama de Classe

A multiplicidade define quantas instâncias de uma classe podem estar associadas a uma instância de outra classe, sendo crucial para especificar as regras de negócio e o comportamento do sistema. Ela é representada por notações específicas em cada extremidade de um relacionamento.

1

Exatamente Um (1)

Uma instância da primeira classe se relaciona com exatamente uma instância da segunda classe.

0

Zero ou Um (0..1)

A associação é opcional; pode haver zero ou uma instância relacionada.



Um ou Mais (1..*)

Uma instância da primeira classe se relaciona com uma ou várias instâncias da segunda classe.



Zero ou Mais (0..*)

A associação pode ter zero, uma ou várias instâncias relacionadas, sendo totalmente flexível.

Associação no Diagrama de Classe

A associação é o tipo de relacionamento mais comum entre classes, indicando como instâncias de uma classe estão conectadas a instâncias de outra, permitindo a comunicação e colaboração entre elas.



Conexão Essencial

Define uma relação estrutural onde objetos de uma classe interagem com objetos de outra para realizar funcionalidades específicas do sistema.



Direcionalidade

Pode ser unidirecional (uma classe acessa a outra) ou bidirecional (ambas se conhecem e interagem), dependendo do fluxo de informação e controle.



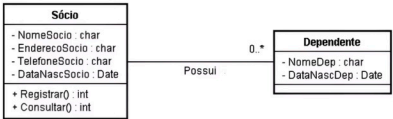
Papéis e Nomes

As extremidades da associação podem ter "nomes de papel" que descrevem a função da classe naquela relação, além de um nome opcional para a própria associação.



Multiplicidade Integrada

É complementada pela multiplicidade, que indica o número de instâncias de uma classe que podem se relacionar com uma instância da outra.



➤ Um Objeto da classe sócio pode ou não relacionar-se com instâncias da classe Dependente
➤ Enquanto que, se existir um objeto da classe Dependente ele terá que obrigatoriamente relacionar-se com um objeto da classe Sócio

Agregação no Diagrama de Classe

A agregação é um tipo especial de associação que representa uma relação "parte-todo" mais fraca, onde as partes podem existir independentemente do todo. É essencial para modelar cenários onde a vida útil dos componentes não está diretamente ligada à do objeto principal.

Relação Parte-Todo

Indica que uma classe é composta por outras classes, mas os componentes mantêm sua própria existência fora do objeto agregado.

Independência dos Componentes

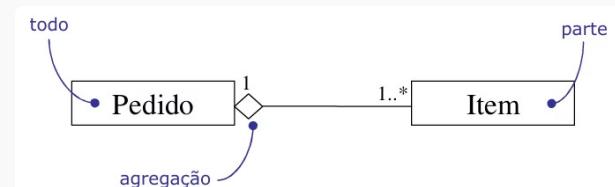
Ao contrário da composição, se o "todo" for destruído, as "partes" podem continuar a existir e ser utilizadas por outros objetos.

Acoplamento Flexível

Oferece maior flexibilidade no design do sistema, pois permite que os componentes sejam compartilhados entre múltiplos agregados.

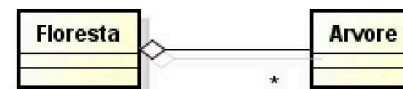
Notação

Representada por uma linha que conecta as classes, com um losango vazio no lado da classe que representa o "todo".



Todo-Parte

- Agregação



Composição no Diagrama de Classe

A composição é um tipo de associação forte de "parte-todo", onde as classes componentes não podem existir independentemente da classe que as agrega. É uma forma de agregação que implica uma dependência de ciclo de vida e exclusividade rigorosas.

Dependência de Ciclo de Vida

A vida da parte está intrinsecamente ligada à vida do todo. Se o objeto "todo" é destruído, os objetos "parte" também são destruídos.

Exclusividade da Parte

Cada parte pertence exclusivamente a um único "todo". Uma instância da classe "parte" não pode ser compartilhada por múltiplas instâncias da classe "todo".

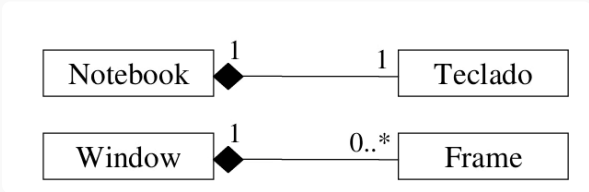
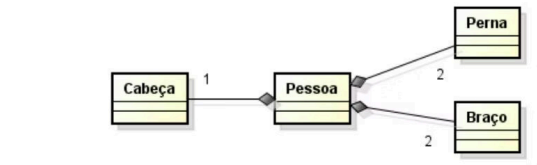
Relação Forte

Representa uma relação "contém" ou "é composto por", onde a parte é um componente integral e essencial para a funcionalidade do todo.

Notação

No diagrama, é indicada por uma linha que conecta as classes, com um losango **preenchido** (sólido) no lado da classe que representa o "todo".

• Composição



Generalização (Herança) no Diagrama de Classe

A generalização, comumente conhecida como herança, é um conceito fundamental em diagramas de classe UML que representa uma relação "é um tipo de" entre classes. Ela permite que uma classe (subclasse ou classe filha) herde atributos e métodos de outra classe (superclasse ou classe pai), promovendo a reusabilidade de código e a organização hierárquica do sistema.

Reusabilidade de Código

Permite que as subclasses reutilizem o comportamento e as propriedades definidas na superclasse, evitando duplicação e facilitando a manutenção.

Hierarquia de Classes

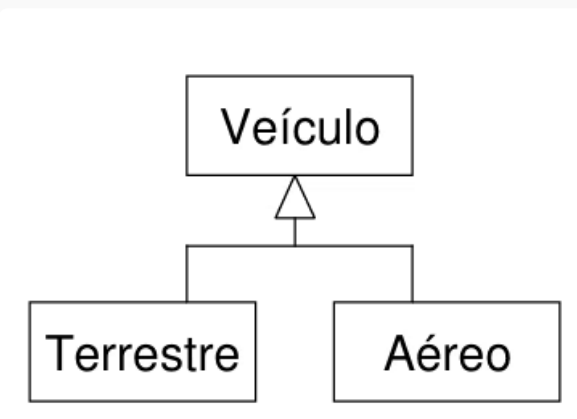
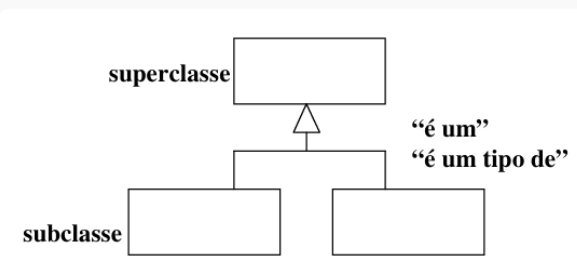
Estabelece uma estrutura organizada onde classes mais específicas derivam de classes mais gerais, refletindo a estrutura do domínio do problema.

Polimorfismo

Viabiliza que objetos de diferentes subclasses sejam tratados como objetos de sua superclasse, permitindo flexibilidade e extensibilidade no design.

Especialização

As subclasses podem adicionar novos atributos e métodos, ou sobrescrever os existentes, para fornecer funcionalidades específicas que as diferenciam da superclasse.



Interpretação de um Diagrama de Classe

Para interpretar um Diagrama de Classe de forma eficaz, é fundamental analisar os seus componentes e as conexões entre eles, revelando a lógica e a estrutura do sistema que está sendo modelado.



Identifique Classes e Membros

Comece localizando as classes e seus nomes, atributos e métodos para entender as entidades principais e suas responsabilidades.



Analise os Relacionamentos

Examine as linhas e setas para decifrar como as classes interagem: associações (gerais ou direcionadas), herança, agregação e composição.



Compreenda a Multiplicidade

Preste atenção aos números e símbolos nas extremidades das linhas de relacionamento para saber quantas instâncias de cada classe podem se associar.

Praticando Interpretação

Cada caixa retangular é uma **Classe**, que funciona como um "molde" para criar objetos.

Dentro de cada caixa temos os atributos e métodos.

Cada linha representa como as classes interagem entre si.

Herança (Seta com Triângulo Vazio) [especificação]

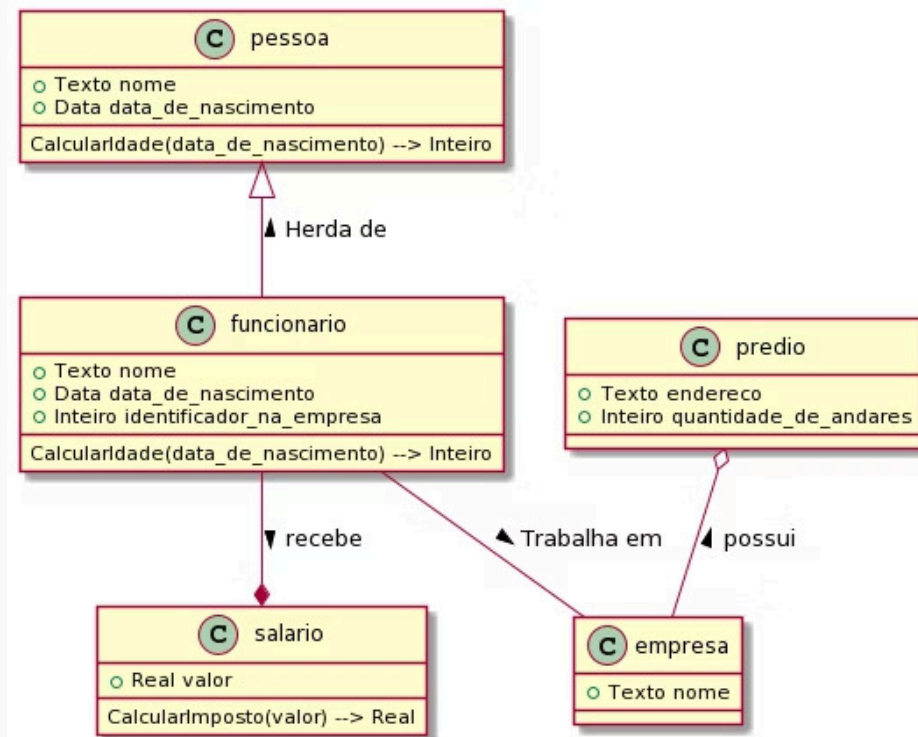
Exemplo: (funcionário é um tipo de pessoa)

Composição (Linha com Losango Preto) ["têm" forte]

Exemplo: (O funcionário "é dono" do salario e salário não existe sem dono)

Agregação (Linha com Losango Vazio) ["têm" fraco]

Exemplo: (A empresa "possui" um prédio, mas eles são independentes. Portanto o prédio pod existir sem uma empresa)



O Que São Metodologias Ágeis?

As Metodologias Ágeis representam uma abordagem revolucionária no gerenciamento de projetos, especialmente no desenvolvimento de software. Elas priorizam a flexibilidade, a entrega contínua e a colaboração, visando responder rapidamente às mudanças e fornecer valor de forma eficiente.

Desenvolvimento Iterativo

Projetos divididos em ciclos curtos (iterações ou "sprints"), permitindo feedback constante e adaptação. O foco é entregar pequenas partes do produto funcionando regularmente.

Colaboração Intensa

Valoriza a comunicação face a face com o cliente e a interação contínua entre os membros da equipe. A transparência e o alinhamento são fundamentais para o sucesso.

Flexibilidade à Mudança

Em um ambiente dinâmico, a capacidade de responder a novos requisitos ou prioridades é crucial. Metodologias Ágeis abraçam a mudança como uma vantagem competitiva.

Entrega de Valor

Prioriza a entrega de software funcional e de alta qualidade em intervalos regulares. Isso garante que o cliente veja o progresso e receba valor desde as fases iniciais do projeto.

Metodologias Ágeis: Scrum e Kanban

As Metodologias Ágeis são abordagens iterativas e flexíveis para o desenvolvimento de software, que priorizam a entrega contínua de valor, a colaboração da equipe e a adaptação rápida a mudanças. Duas das mais populares são Scrum e Kanban.



Scrum

Um framework para gerenciar projetos complexos, conhecido por seus ciclos de trabalho curtos e iterativos (Sprints), papéis bem definidos (Product Owner, Scrum Master, Time de Desenvolvimento) e eventos regulares (Daily Scrum, Review, Retrospective).



Kanban

Um método para gerenciar e melhorar o trabalho, visualizando o fluxo de tarefas, limitando o trabalho em progresso (WIP) e focando na entrega contínua. É ideal para otimizar fluxos de trabalho existentes e identificar gargalos.

Kanban em Ação: Aplicando a Metodologia a Tarefas

Aplicar Kanban no desenvolvimento de tarefas transforma a gestão do trabalho em um processo visual, eficiente e adaptável. Ao seguir passos claros, equipes podem otimizar o fluxo, reduzir gargalos e entregar valor continuamente.

Visualizar o Fluxo de Trabalho

Crie um quadro Kanban com colunas que representem as etapas do seu processo (Ex: A Fazer, Em Andamento, Em Revisão, Concluído). Cada tarefa é um "cartão" que se move pelas colunas.

Gerenciar o Fluxo

Monitore o movimento dos cartões pelo quadro. O objetivo é que as tarefas fluam da esquerda para a direita o mais rápido e suavemente possível, identificando e resolvendo impedimentos.

Limitar o Trabalho em Progresso (WIP)

Defina um número máximo de tarefas que podem estar "Em Andamento" simultaneamente. Isso força a equipe a focar na conclusão de itens, em vez de iniciar muitos ao mesmo tempo.

Melhorar Continuamente

Analise métricas como tempo de ciclo (lead time) e taxa de transferência para identificar oportunidades de otimização. Realize reuniões regulares para discutir melhorias no processo.

Atividade

1. Pensar nos passos necessários para montar um diagrama de classe da clínica com base nas suas características.
2. Organizar esses passos em um quadro Kanban, seja no GitHub mesmo ou no Trello (preciso do link para conseguir visualizar os cartões referentes a cada tarefa).
 - a. Exemplo de um banco:
 - i. montar a classe banco
 - ii. montar a classe conta corrente
 - iii. montar a classe cliente
 - iv. montar a relação de cliente com conta corrente
3. Montar o diagrama de classe do sistema da clínica com base nas tarefas definidas.
4. Colocar no GitHub:
 - a. Imagem do diagrama
 - b. um arquivo MD com o link da organização